Breadth-First Search

Level-by-level graph and tree traversal (BFS)

Overview

Breadth-First Search (BFS) explores a graph or tree one level at a time. Starting from a source node, it visits all of the source's immediate neighbors before moving on to their neighbors, and so on. BFS uses a First-In-First-Out (FIFO) queue to keep track of which node to visit next, which guarantees that nodes closer to the start are explored before nodes farther away.

Algorithm Steps

1. Add the start node to an empty queue and mark it as visited.
2. While the queue is not empty, remove (dequeue) the front node.
3. Process the removed node (e.g., check if it is the goal).
4. Enqueue all unvisited neighbors of that node and mark them visited.
5. Repeat until the queue is empty or the goal is found.

Complexity

Metric	Value
Time Complexity	O(V + E) for a graph with V vertices and E edges
Space Complexity	O(V) — stores all discovered nodes in the queue/visited set
Completeness	Complete (finds a solution if one exists, on finite graphs)
Optimality	Optimal when all edge costs are equal (e.g., unweighted graphs)

Advantages & Disadvantages

Advantages

✓ Guarantees the shortest path in unweighted graphs
✓ Simple to implement with a queue
✓ Explores all nodes at the current depth before going deeper

Disadvantages

✗ Can use a large amount of memory for wide graphs
✗ Slower than DFS when the goal is deep in the tree
✗ Not ideal for very large search spaces

Typical Use Cases

- Finding the shortest path in an unweighted graph or maze
- Web crawlers exploring pages level by level
- Finding connected components in a network

- Peer-to-peer networks and broadcasting (e.g., GPS navigation systems)

C++ Implementation

C++

```cpp
#include <queue>
#include <vector>
using namespace std;

void bfs(int start, vector<vector<int>>& adj, int n) {
    vector<bool> visited(n, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();
        cout << node << " ";  // process node

        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```